Targeted C++ practice - study priorities

Revised to separate explicit TA-note priorities from supplementary revision. These are original practice exercises, not confirmed exam questions.

Part A: TA-emphasized topics - do these first

A1 Function pointers and callbacks (page 2). A2 Operator overloading, const and return types (page 3). A3 Abstract interfaces and virtual functions (page 4). A4 Writing and tracing function templates (page 5).

Evidence: your C++ Concept Review.md notes explicitly emphasize writing function pointers, understanding virtual/abstract interfaces, writing operators and explaining const, and writing/tracing function templates. The specific exercises and callback counting task were designed for practice.

Part B: supplementary - exam inclusion not confirmed

B1 Const pointers; B2 Constructor initialization/static state; B3 Lifetime; B4 Array bounds; B5 Strings; B6 Ownership; B7 Stream formatting; B8 Stream recovery; B9 Optional std::vector/iteration (pages 6-14).

These address your questions and earlier mistakes, or broader lecture material. Some support Part A and some appear in tutorial material, but that does not confirm their exam inclusion. This label does not mean the TA excluded them.

If time is short, finish A1-A4 first. Use B1-B2 when const or initialization blocks your understanding of A2. Treat B9 as optional: Lecture 9 uses capital-V Vector examples; std::vector<int> was my supplementary exercise choice.

How to use this booklet

Assume C++17, required standard headers, and using namespace std unless qualified. No numerical memory addresses are requested. Write complete code and explain requested reasoning. Questions and solutions use matching A/B identifiers. This is targeted practice, not a timed or scored mock exam.

Sources for the distinction: C++ Concept Review.md (your TA notes); tutorial review photographs as summarized in 2026-10-26_Tutorial_Review.md; Lectures/9-CSC3002_Template and Iterator.pdf (broader iteration coverage).

A1. Function pointers: pass a rule into a loop

```
bool isPositive(long x) { return x > 0; }
bool (*test)(long) = isPositive;

int countIf(const long* a, int n,
            bool (*pred)(long)) {
    int count = 0;
    for (int i = 0; i < n; ++i) {
        if (pred(a[i])) ++count;
    }
    return count;
}

bool isEven(long x) { return x % 2 == 0; }
long data[] = {-3, 0, 4, 7};
// countIf(data, 4, test)   returns 2
// countIf(data, 4, isEven) returns 2
```

isPositive names the function and converts to a function pointer in this initialization. isPositive(5) calls the function and yields a bool. test and pred are pointer variables; count is the integer counter. pred(a[i]) calls the chosen function on one element. (*pred)(a[i]) is equivalent. Zero is even.

A2. Money operators: types, const and *this

```
struct Money {
    int cents;
    Money operator+(const Money& rhs) const {
        return Money{cents + rhs.cents};
    }
    Money& operator+=(const Money& rhs) {
        cents += rhs.cents;
        return *this;
    }
};
```

(b) cents is int; rhs is a Money object accessed through a const reference. int + Money has no matching operator in this class. cents + rhs.cents is int, but this class has no implicit conversion from int to Money. Money{...} initializes a new Money with the sum.

In +=, this points to the left object and *this denotes that object. Returning *this with return type Money& returns a reference to the same object, allowing another += to modify it. It is not a reference to a newly created local result.

(c) a = 100, b = 15, c = 5, d = 95. Money const as a return type qualifies the returned value. Trailing const qualifies the member function and allows calls on const left operands. These are not interchangeable.

A3. Abstract interfaces and exact overrides

```
struct QuietReporter : public Reporter {
    int report() const override { return 0; }
    int tag() const { return 9; }
};
```

(a) Reporter has a pure virtual report and is abstract. A shared interface lets callers use different derived implementations through the same base reference or pointer.

(b) The results are 0, 1, 9. report is virtual: r.report() chooses the final override for the dynamic object, QuietReporter. tag is nonvirtual: r.tag() uses Reporter::tag, while s.tag() uses QuietReporter::tag.

(c) const int report() places const on the return type and lacks the required trailing const. It does not provide the required override; adding override exposes the mismatch as an error. The virtual destructor supports correct destruction through the base pointer. Derived methods cannot directly access private base members merely because they inherit from the base.

A4. Function templates: write, deduce and trace

```
template<typename T>
T third(T x) { return x / 3; }
```

(a) a: int, 2. b: double, 3.0. c: double, 2.0. d: double, 3.0. For c, int division truncates before the result is converted to double. An explicit double template argument converts the integer argument before the function arithmetic.

```
larger<double>(2, 2.5); // returns 2.5

template<typename T>
T addOne(T x) { return x + 1; }
```

(b) Deduction proposes conflicting types int and double for the same T. Explicitly choosing double permits conversion of the int argument. (c) addOne(4) returns int 5; addOne(1.5) returns double 2.5. third(8) performs int division first; converting its result to double produces 2.0, not the original fraction.

B1. Const pointers: what is protected?

(a) Valid, invalid, invalid, valid, valid. p points to const int: modification through p is forbidden, but p can be reseated. q is a const pointer to mutable int: q cannot be reseated, but *q can be modified. a itself is not const.

```
const int* const r = &a;

int readValue(const int* p) {
    return *p;
}
```

(b) Both pointer and pointed-to access are const. (c) The caller may still assign to a directly. const access through one pointer does not make the original non-const object immutable.

B2. Constructor initialization and static state

```
struct Ticket {
    const int code;
    static int created;
    Ticket(int c) : code(c) { ++created; }
    int getCode() const { return code; }
};
int Ticket::created = 0;
```

(b) code(c) initializes the data member code from parameter c before the constructor body runs. It does not call a function named code. Assignment in the body is too late for a const member; initialization and assignment are different operations.

(c) The values are 9, 14, and 2. created is one shared static member. getCode returns int, matching the type of code; trailing const qualifies the member function, not its return type.

B3. Object lifetime and dangling pointers

(a) value is an automatic local object. Its lifetime ends when makeBad exits. q receives a pointer to that expired object, not a copy of the int. q itself can still exist while its pointee is dead: that makes it dangling. Dereferencing it has undefined behavior. A compiler may warn; this need not be rejected as a compilation error.

```
int makeGood() {
    int value = 11;
    return value;
}
// In main:
int answer = makeGood();
cout << answer; // 11

int* identity(int& x) { return &x; }
// In main:
int n = 11;
int* p = identity(n);
cout << *p; // 11
```

(c) x refers to the caller's n; it does not create a separate local int. Returning from identity does not end n's lifetime. The pointer remains usable while n remains alive. A pointer never extends the lifetime of its target by itself.

B4. Array parameters and one-past boundaries

```
int sum(const int a[], int n) {
    int total = 0;
    for (int i = 0; i < n; ++i) total += a[i];
    return total;
}

int values[] = {2, 4, 6, 8};
int* end = values + 4;
int total = 0;
for (const int* p = values; p != end; ++p) {
    total += *p;
} // total is 20
```

(a) An array parameter is adjusted to a pointer parameter. sizeof(a) is therefore the size of a pointer, not the original array. The explicit n gives the loop its bound.

(b) Forming the one-past pointer is valid. Reading *end has undefined behavior. Reading *(end - 1) is valid and yields 8. The loop in (c) tests before dereferencing, so it never reads through end.

B5. C-strings: capacity, terminators and concatenation

(a) sizeof(label) = 3, strlen(label) = 2, and Blue needs capacity 5 including the null terminator. Copying it into label overflows the three-element array.

```
char safe[5] = "Blue";

string text = "Go";
text = "Blue";
```

(c) "Hi" + "!" is invalid: there is no operator+ joining two string literals this way. string("Hi") + "!" and "Hi" + string("!") are valid and yield "Hi!". strcmp("Go", "Go") returns zero, indicating equal contents. strcpy copies characters into existing storage; it does not allocate or resize the destination.

B6. Ownership: raw aliases and smart pointers

(a) p is null; q dangles. Reading *q or deleting q again has undefined behavior. Assigning nullptr to p does not change the separate pointer variable q.

```
int* p = new int(6);
int* q = p;
delete p;
p = nullptr;
q = nullptr;

auto s = make_shared<int>(6);
weak_ptr<int> w = s; // use_count is 1
if (auto owner = w.lock()) {
    cout << *owner;
} // temporary strong owner ends here
s.reset(); // w.expired() is now true

auto first = make_unique<int>(6);
auto second = std::move(first);
if (!first) cout << "empty";
```

(b) weak_ptr does not add a strong owner. lock obtains a shared_ptr if the object is still alive. (c) unique_ptr ownership is moved, not copied. new int[1] requires delete[] even though there is only one element. Copying a raw pointer copies the pointer value; it does not allocate or copy the pointee.

B7. Streams: exact output and persistent formatting

```
18:[]
[Lin Chen]
___4.50|8.00
```

(a) Formatted integer extraction leaves the newline unread. The first getline consumes that delimiter and returns an empty string. 4.50 has four characters, so width 7 adds three underscores.

(b) This numeric insertion resets width. Fill, fixed, and precision persist. out.str() returns the string contents; the getter alone neither prints nor clears the buffer.

```
in >> id;
in.ignore(numeric_limits<streamsize>::max(), '\n');
getline(in, name);
```

(c) Apply this replacement to a fresh stream with the original input. Include <limits> for numeric_limits. ignore consumes the rest of the id line including its newline; getline then reads the complete name.

B8. Recover from failed numeric input

(a) total is 20. Extraction encounters x, fails, and the loop condition is false. x remains unread; the loop body is not entered for the failed extraction.

```
numbers.clear();
char bad;
numbers >> bad;
if (numbers >> n) {
    total += n;
}
// total is 45
```

(b) clear resets error flags, and extraction into char consumes x (skipping preceding whitespace). The guarded integer read then adds 25.

(c) clear does not remove unread input. Without consuming x, the next integer extraction fails again. sizeof(n) reports the storage size of type int, which is unrelated to the characters in the stream and is not universally four bytes. Casting an int to int does not reset stream state or remove x.

B9. Optional standard-library vector and iteration

```
for (auto& x : v) x += 3;
// v becomes {5, 8, 11}

// With a fresh v{2, 5, 8}:
for (auto it = v.begin(); it != v.end(); ++it) {
    cout << *it << ;
}

int values[] = {2, 5, 8};
for (int* p = values; p != values + 3; ++p) {
    cout << *p << ;
}
```

auto& refers to the element; auto alone makes a copy. end() and values + 3 denote one-past positions. Compare against them, but do not read through them. Both fresh traversals print 2 5 8. This exercise does not establish standard-library vector as an exam topic.